

petta-memory: Project Status, Implemented Work, and Proposed Directions

Prepared for Benjamin Goertzel

10 August 2026

Executive summary

petta-memory is a local-first Python/MeTTa prototype for a bounded, symbolic intermediate memory layer between volatile agent context and broad vector/Markdown long-term memory. Its original core is an append-only journal of complete **MemoryCluster** records with deterministic retrieval and separate audit, prompt, index, and PLN-safe views. The project subsequently expanded into provenance-preserving evidence packets, reversible pi-PLN projections, patham9/PLN compilation and replay, PeTTaChainer diagnosis and bounded repaired-runtime experiments, GoalChainer handoff gates, and provider-free usability/shadow-consumer evidence.

Observed baseline. The prepared takeover worktree is at commit `5b842f4d8e203d86c0d14f42eb91a03535376c0a` on branch `agent/protocosmo2-handoff`. The source project recorded a clean 698-test run at this baseline. A fresh run for this report collected all 698 tests but finished with one failure, one error, and eight skips because the isolated worktree's relative dependency paths do not resolve to the existing PeTTaChainer/patham9 sibling repositories. This is a reproducibility defect in the handoff layout, not evidence that the affected semantics regressed. The branch contains 352 commits not present in its configured upstream branch. Consequently, most recent implementation is local and must not be described as published to GitHub until it is pushed.

Overall assessment. A substantial, heavily tested research prototype exists. It is not yet a production agent-memory service. The safest next step is to freeze the present boundary, publish the accumulated history on a review branch, run one reproducible end-to-end takeover canary, and then choose one narrow frontier rather than continuing unbounded defensive hardening.

1 Purpose and design principles

The intended role is a medium-duration symbolic store for ProtomegaTron, OmegaClaw, or related agents. The main design principles discussed and adopted are:

- append complete, delimited records and preserve provenance;
- distinguish observations, quoted claims, derived beliefs, hypotheses, decisions, commitments, boundaries, status changes, and artifacts;
- keep raw utterances and unpromoted claims out of factual PLN premises;
- make indexes, prompt context, attention, and PLN projections rebuildable views rather than canonical truth stores;
- attach explicit evidence counts and truth-value provenance rather than reconstructing evidence from an STV;
- make inference and live-agent integration bounded, reviewable, fail-closed, and non-promoting by default;
- preserve modular seams so patham9/PLN, PeTTaChainer, ECAN, GoalChainer, or future Hyperon substrates can be replaced independently.

2 What has actually been implemented

2.1 Core journal and views

- A Python package, `petta_memory`, with a `MediumMemoryStore` append-only `.metta` journal.
- Complete `MemoryCluster` envelopes, schema/version checks, S-expression parsing, size and identifier bounds, locking, duplicate-ID rejection, provenance requirements, and caller-supplied PeTTa parse checking.
- Query by cluster/id, type, `About`, status, and epistemic role; status is reconstructed from append-only status and supersession events.
- Bounded canonical audit, index, prompt, PLN-safe, PeTTaChainer statement, `EvidencePacket`, `patham9` handoff, and `GoalChainer` handoff views.
- A CLI covering append, query, tail, views, handoff caches, smoke gates, profiles, and bridge operations.

2.2 OmegaClaw and GoalChainer boundaries

- An inert OmegaClaw bridge policy: reads require explicit feature flags and autonomous writes are rejected.
- Non-live GoalChainer subprocess smoke gates and a read-only live bridge that validates promoted evidence and copied decision metadata.
- Extensive rejection of directive/task-claim sidecars, malformed semantic markers, count drift, and schema confusion before a bridge artifact is admitted.
- No general live autonomous write path has been enabled.

2.3 pi-PLN evidence and replay substrate

- Immutable typed evidence tokens, evidence bases, `EvidencePackets`, snapshots, contexts, chart policies, pi-charts, compiler inputs, results, captures, and manifests.
- Exact union-by-basis packet algebra, conflict checks, content-derived fingerprints, beta projection/prior cycling, and compatibility-labelled legacy EC projection.
- Deterministic `patham9` query-program construction, bounded shell-free subprocess capture, executable and input limits, exact result admission, create-once artifacts, clean-room reload, and semantic replay comparison.
- Filesystem and subprocess hardening for symlinks, permissions, artifact identity, bounded streams, process groups, timeouts, cleanup failures, and malformed reconstructed objects.

2.4 PeTTaChainer work

- Typed checked-add/query contracts adapted from compiler output while retaining stamps and evidence bases as audit sidecars.
- Profilers that localized severe fact compilation fan-out in pinned PeTTaChainer source: duplicate imports and overlapping dispatch/conversion paths were measured under exact source gates.
- An isolated duplicate-import candidate reduced a 128-copy direct dispatch to one normalized-equivalent clause. Bounded experiments then completed `mm2compile`, exact `compileadd`, stored-fact retrieval, and a one-rule derivation.
- The admitted one-rule experiment derived $(T \ a)$ from $(S \ a)$ and $S \rightarrow T$; its STV matched the pinned `TotalMpFormula` result (STV 0.7600000000000001 0.52).
- Typed, non-promoting derived-result captures and PeTTaChainer-specific episode manifests were persisted with content and provenance commitments.

2.5 Evidence and verification

The repository contains 12 Python package files and 11 test files. Project records report 698 passing tests at the handoff baseline; the fresh isolated worktree run described above did not reproduce a clean pass because two tests used unresolved relative dependency paths. The two largest test areas are patham9/PLN and PeTTaChainer profiling, followed by pi-PLN models. Stored experiment ledgers cover provider-free usability round trips and ProtoMegaBot2 shadow-source/consumer bundles. Checked-in artifacts include GoalChainer bridge probes and bounded PeTTaChainer derivation captures.

3 Plans and ideas discussed or proposed

The following items are architectural proposals, partial roadmaps, or future gates. They must not be confused with deployed capabilities.

1. **Production agent integration.** Connect the store to ProtoCosmo2/OmegaClaw for routine prompt retrieval and eventually reviewed writes, with exactly one writer and explicit authority boundaries.
2. **Atlas-indexed reversible pi-PLN.** Treat evidence as immutable packets, project it into local charts, compile bounded episodes, preserve complete provenance, and allow replay/retraction without confusing chart priors with empirical evidence. Much substrate exists, but unrestricted production inference and belief promotion remain closed.
3. **PeTTaChainer repair/adoption.** Decide whether to upstream the duplicate-import repair, maintain a pinned local patch/profile, or use patham9 as the primary backend. The experiments establish a narrow repaired path, not general semantic correctness across PeTTaChainer workloads.
4. **ECAN attention projection.** Use metta-attention/ECAN as a derived, disposable salience view over canonical memory and OmegaSelf, never as the truth store or authority mechanism.
5. **GoalChainer/OmegaClaw task appraisal.** Supply promoted evidence as read-only appraisal context, later considering tightly bounded task claims only after separate review. Current code intentionally rejects task/directive artifacts.
6. **Two-strata memory.** Combine an append-only epistemic record with compact operational/index layers, preserving the former while permitting the latter to be rebuilt or optimized.
7. **Provider-free portability.** Package source, journals, views, inference summaries, and read-only canaries so a different agent can validate usefulness without privileged provider access.
8. **Further inference control.** Ranked branch plans, estimator and continuation-predicate wrappers, controller-as-chainer composition, bounded derive gates, and eventually meta-learning of inference control.
9. **General runtime attribution.** Extend beyond exact stored-fact and one-rule cases to trustworthy rule/trace attribution, broader corpus coverage, and semantic-failure classification.

4 Explicitly not implemented or not authorized

- No production-quality multi-agent memory service or daemon.
- No unrestricted autonomous journal writer for ProtoCosmo2/OmegaClaw.
- No general inferred-belief promotion from a runtime answer into canonical memory.
- No claim that all PeTTaChainer compilation/query semantics are repaired.
- No upstream PeTTaChainer patch publication recorded by this project.
- No general proof-trace identity for arbitrary patham9 or PeTTaChainer episodes.
- No paid compute, external authority expansion, or credential material in the project files.

5 Current risks and technical debt

- **Publication gap:** 352 local commits are ahead of the configured remote tracking branch. Loss of the laptop or repository would lose work not present remotely.
- **Scope accretion:** core store, inference research, runtime repair, bridge security, and artifact hardening coexist in large modules. In particular, `patham9_pln.py`, `pettachainer_profile.py`, and `pipln_models.py` are each large and merit modular decomposition.
- **Recorded-state drift:** project Markdown accumulated long chronological checklists, including duplicated status material. Code, tests, commit history, and experiment records should outrank prose when they conflict.
- **Dependency pinning:** runtime results depend on exact local `patham9`, `PeTTa`, `PeTTaChainer`, `SWI-Prolog`, and `GoalChainer` trees. A takeover must record their commits before repeating runtime claims.
- **Over-hardening risk:** recent work concentrated on increasingly narrow malformed-object and cleanup cases. The July 22 decision already recommended freezing generic hardening and moving to a bounded semantic capture/reload gate.

6 Recommended takeover sequence

1. Preserve the frozen `5b842f4` worktree and rerun the 698-test suite.
2. Record exact dependency commits and execute the provider-free usability gate as the first independent canary.
3. Review the 352-commit publication gap, scan for secrets and oversized artifacts, then push `agent/protocosmo2-handoff` (not the default branch) and open a draft PR or review branch.
4. Produce a small architecture split proposal for the three oversized modules without changing semantics.
5. Choose one frontier: (a) production read-only retrieval canary, (b) broader repaired `PeTTaChainer` semantic corpus, or (c) `patham9`-backed bounded inference/replay. State the acceptance test before coding.
6. Keep journal writes and inferred-belief promotion disabled until a separate reviewed experiment demonstrates provenance, idempotence, rollback, and single-writer behavior.

Provenance of this report

This summary was checked against the local project records `PROJECT.md`, `TASKS.md`, `DECISIONS.md`, `NOTES.md`; repository `README`, source tree, tests, Git history and remote configuration; and recorded experiment runs. It distinguishes local implementation from remote publication and bounded experiment evidence from general deployment claims. No credential values were read into or written to this document.